

RISC-V CPU Core Documentation — Concept-First Deep Dive

Generated from HDL Project Analysis

June 6, 2026

Contents

1	High-Level Overview	6
1.1	What this CPU is	6
1.2	Why there are multiple CPU files	6
1.3	Supported ISA subset, at a glance	7
2	Repository / File Structure	8
3	CPU Architecture Overview	12
3.1	The basic CPU loop	12
3.2	Main conceptual block diagram	12
3.3	Single-cycle vs multi-cycle vs pipelined	12
3.3.1	Single-cycle	13
3.3.2	Multi-cycle	13
3.3.3	Pipelined	13
4	Instruction Encoding and Decode	14
4.1	Why decode exists	14
4.2	How decode is implemented here	14
4.3	Instruction families	15
4.3.1	R-type integer operations	15
4.3.2	I-type ALU operations	15
4.3.3	Loads and stores	15
4.3.4	Branches and jumps	15
4.3.5	System instructions	15
5	Immediate Generator	16
5.1	Why immediates look strange in RISC-V	16
5.2	Immediate formats	16
5.3	Why B and J immediates end in zero	16
5.4	Implementation role	16

6	Register File	18
6.1	What the register file is	18
6.2	The special rule for x0	18
6.3	Read and write timing	18
7	ALU and RV32M Multiply/Divide	19
7.1	Why the ALU exists	19
7.2	Base RV32I ALU operations	19
7.3	Signed vs unsigned behavior	19
7.4	RV32M multiplication	19
7.5	RV32M division and remainder	20
7.6	Hardware cost	20
8	Program Counter, Branches, and Jumps	21
8.1	What the PC does	21
8.2	Conditional branches	21
8.3	8.3 jal	21
8.4	8.4 jalr	21
8.5	Control hazard in a pipeline	22
9	Load/Store Unit and Memory Access	23
9.1	Why RISC-V is load/store	23
9.2	Effective address calculation	23
9.3	Load sizes and sign extension	23
9.4	Store sizes and byte masks	23
9.5	Memory-mapped I/O	24
10	CSR, Privilege, Exceptions, and Traps	25
10.1	Why CSRs exist	25
10.2	What a trap is	25
10.3	Privilege model in this design	25
10.4	Supported CSRs	25
10.5	CSR instructions	26
10.6	10.6 Trap entry and mret	26
10.7	Illegal instruction detection	27
11	Timer and CLINT	28
11.1	Why a timer matters	28
11.2	Machine timer interrupt flow	28
11.3	CLINT-style block	28
12	Atomic Operations	29
12.1	Why atomics exist	29
12.2	RISC-V atomic concepts	29
12.3	Implementation in this repository	29
12.4	Important limitation	29

13 MMU and Virtual Memory	30
13.1 The problem: software should not always use physical addresses	30
13.2 Virtual address vs physical address	30
13.3 What a page is	30
13.4 What a page table is	30
13.5 Why multi-level page tables exist	31
13.6 Now Sv32: the RISC-V 32-bit virtual memory scheme	31
13.7 13.7 satp: where translation begins	31
13.8 PTE format and permissions	32
13.9 Page-table walk	32
13.10How physical addresses are formed	32
13.11When translation is active	33
13.12Page faults	33
13.13Two MMU implementations in this project	33
13.13.1 rtl/mmu.v — combinational educational MMU	33
13.13.2 rtl/cpu_mc_mmu.v — synthesizable multi-cycle walker	33
13.14What is simplified	34
14 Pipeline, Hazards, Forwarding, and Stalls	35
14.1 Why pipelining exists	35
14.2 Pipeline registers	35
14.3 Data hazards	35
14.4 Forwarding	35
14.5 Load-use stall	36
14.6 Control hazards and flushing	36
15 Precise Traps in the Pipeline	37
15.1 Why traps are harder in a pipeline	37
15.2 What precise means	37
15.3 15.3 Implementation in cpu_pipe_trap.v	37
15.4 Why EX-stage trap commit is a reasonable teaching choice	37
16 Branch Prediction	38
16.1 Why prediction exists	38
16.2 What must be predicted	38
16.3 BHT: branch history table	38
16.4 BTB: branch target buffer	38
16.5 Training the predictor	38
16.6 16.6 Integration in cpu_pipe_bp.v	39
16.7 Performance counters	39
17 SoC Integration and Memory Map Concept	40
17.1 Why a CPU core needs a SoC wrapper	40
17.2 Memory-mapped peripherals	40
17.3 Important wrappers	40

18 UART, Debug, and System Control	41
18.1 Why UART is useful	41
18.2 Debug support	41
18.3 System control	41
19 FPGA and Synthesis Notes	42
19.1 Why simulation code and FPGA code differ	42
19.2 FPGA top modules	42
19.3 Practical synthesis concerns	42
20 Software and Verification	43
20.1 Why testbenches matter	43
20.2 What to test per block	43
20.3 Software demos	43
21 How to Read the Codebase	45
22 Design Philosophy and Limitations	46
22.1 Design philosophy	46
22.2 Known simplifications	46
23 Suggested Future Improvements	47
24 Summary	48

Preface

This document explains the CPU core as an engineering design, not only as a list of modules. Each major part follows the same pattern:

1. **Why the block exists** — the CPU concept.
2. **What RISC-V requires** — the architectural meaning.
3. **How this repository implements it** — the actual HDL modules and signals.
4. **Design limits and trade-offs** — what is simplified, educational, or synthesis-oriented.

The repository is best understood as a **family of progressively extended RV32 cores**. It starts with a simple single-cycle RV32I/RV32IM educational core and grows into SoC, multi-cycle, pipelined, trap-capable, MMU-capable, branch-predicting, UART/FPGA, and RTOS-oriented variants.

1 High-Level Overview

1.1 What this CPU is

A CPU core is the hardware block that repeatedly performs this loop:

```
fetch instruction -> decode instruction -> read operands -> execute ->
    access memory if needed -> write result -> choose next PC
```

For a RISC-V CPU, the instructions have a standardized binary encoding. The CPU must interpret those 32-bit instruction words and update architectural state exactly as the RISC-V ISA defines. The two most important pieces of architectural state are:

- the **program counter**, or PC, which says where the current instruction comes from;
- the **integer register file**, x0 through x31, which holds software-visible values.

This project implements that loop in multiple ways. Some versions complete one instruction per clock. Some sequence an instruction over multiple states. Some overlap several instructions in a pipeline.

1.2 Why there are multiple CPU files

This is not one monolithic CPU design. The repository is organized like a learning path:

Design level	Main files	Meaning
Single-cycle educational CPU	rtl/cpu.v	Easiest to understand: one instruction completes per clock.
SoC-oriented single-cycle-style CPU	rtl/cpu_core.v, rtl/soc.v	Data memory is moved outside the CPU so peripherals can be memory-mapped.
Multi-cycle CPU	rtl/cpu_mc.v	More synthesis-friendly for block RAM because memory reads take registered cycles.
Pipelined CPU	rtl/cpu_pipe.v	Higher throughput by overlapping IF/ID/EX/MEM/WB stages.
Pipelined CPU with traps	rtl/cpu_pipe_trap.v	Adds precise exceptions/interrupts to the pipeline.
Pipelined CPU with branch prediction	rtl/cpu_pipe_bp.v, rtl/branch_predictor.v	Reduces taken-branch penalty using prediction.

Design level	Main files	Meaning
MMU variants	<code>rtl/mmu.v</code> , <code>rtl/cpu_core_mmu.v</code> , <code>rtl/cpu_mc_mmu.v</code> , <code>rtl/soc_mmu.v</code>	Adds virtual-to-physical data address translation experiments.
FPGA/RTOS/debug variants	<code>rtl/fpga_top*.v</code> , <code>rtl/soc_rtos*.v</code> , <code>rtl/debug_module.v</code>	Integration wrappers for real boards, UART, timers, and RTOS experiments.

The result is useful for documentation because the same architectural idea appears in several forms. For example, the ALU is mostly the same concept in single-cycle, multi-cycle, and pipelined designs, but the timing around it changes.

1.3 Supported ISA subset, at a glance

The source code indicates support for these features in different variants:

Feature	Meaning	Where it appears
RV32I	Base 32-bit integer ISA	Core datapath, <code>control.v</code> , <code>alu.v</code> , <code>immgen.v</code> , <code>regfile.v</code>
RV32M	Multiply/divide extension	<code>alu.v</code> , decoded by <code>control.v</code> when <code>funct7 == 0000001</code>
Zicsr	CSR read/write instructions	<code>csr.v</code> , <code>cpu_core.v</code> , <code>cpu_mc.v</code> , <code>cpu_pipe_trap.v</code> , MMU variants
Machine-mode traps/interrupts	Exceptions, <code>ecall</code> , <code>ebreak</code> , timer/external interrupts, <code>mret</code>	<code>csr.v</code> , trap-capable CPU variants
RV32A-style atomics	LR/SC and AMO read-modify-write operations	<code>cpu_core.v</code> atomic datapath and decode support in <code>control.v</code>
Sv32 virtual memory	32-bit paged virtual memory	<code>mmu.v</code> , <code>cpu_core_mmu.v</code> , <code>cpu_mc_mmu.v</code>
Branch prediction	BTB + 2-bit BHT	<code>branch_predictor.v</code> , <code>cpu_pipe_bp.v</code>

A safe way to describe the project is: **the base core is RV32I/RV32IM-oriented, with selected privileged, atomic, MMU, and prediction features added in specific variants.**

2 Repository / File Structure

The important HDL files are below.

File / Module	Purpose	Important Notes
rtl/alu.v	ALU	Combinational arithmetic/logic unit for RV32I/RV32M-style operations.
rtl/control.v	Decoder / control	Decodes opcode, funct3, funct7 into datapath control and ALU operation selection.
rtl/immgen.v	Immediate generator	Reconstructs I/S/B/U/J immediates from instruction bits and sign-extends them.
rtl/regfile.v	Integer register file	32 x 32-bit GPRs, two combinational read ports, one synchronous write port, hardwired x0.
rtl/imem.v	Instruction memory	Simple instruction ROM/RAM style model for simulation-oriented cores.
rtl/dmem.v	Data memory	Byte/halfword/word load-store memory model for simple cores.
rtl/cpu.v	Single-cycle CPU top	Self-contained CPU with internal instruction and data memories.
rtl/cpu_core.v	SoC CPU core	Single-cycle-style core with external data bus, CSR/trap logic, atomics, and timer interrupt input.
rtl/cpu_mc.v	Multi-cycle CPU	BRAM-ready CPU using FETCH/EXEC/MEM-style sequencing.
rtl/cpu_pipe.v	Pipelined CPU	5-stage pipeline with forwarding, load-use stall, and predict-not-taken flushing.
rtl/cpu_pipe_trap.v	Pipelined CPU with traps	Adds precise CSR/trap behavior to the pipeline.

File / Module	Purpose	Important Notes
rtl/branch_predictor.v	Branch predictor	BTB plus 2-bit saturating counter BHT.
rtl/cpu_pipe_bp.v	Pipelined CPU with prediction	Integrates <code>branch_predictor.v</code> and performance counters.
rtl/csr.v	CSR/trap unit	Minimal machine-mode CSR bank, trap entry, <code>mret</code> , timer/external interrupt pending logic, <code>satp</code> .
rtl/mmu.v	Combinational MMU	Educational Sv32 data-side translator using dedicated page-table read ports.
rtl/cpu_core_mmu.v	CPU + combinational MMU	Single-cycle/SoC-oriented CPU variant with data-side translation.
rtl/cpu_mc_mmu.v	Multi-cycle CPU + MMU	Synthesizable page-table walker using registered memory/data bus sequencing.
rtl/clint.v	Timer/CLINT	Machine timer support for interrupt-driven software/RTOS experiments.
rtl/timer.v	Simple timer	Earlier/smaller timer peripheral.
rtl/uart*.v	UART peripherals	Transmit/receive/configurable UART blocks for FPGA/-SoC integration.
rtl/syscon.v	System control	System-control peripheral, including halt-style control in some SoC variants.
rtl/debug_module.v, rtl/cpu_core_dbg.v, rtl/soc_dbg.v	Debug support	Experimental/debug-oriented CPU and SoC wrappers.
rtl/soc*.v	SoC tops	Memory/peripheral integration around CPU variants.
rtl/fpga_top*.v	FPGA tops	Board-oriented wrappers for synthesis/deployment.
constraints/*.xdc	FPGA constraints	Zynq-related pin/clock constraints.

File / Module	Purpose	Important Notes
sw/*	Software examples	Bare-metal/RTOS/demo code and generated hex files.
tb/*	Testbenches	Simulation checks for CPU blocks and full systems.
docs/*	Tutorial notes	Step-by-step design documentation already present in the repo.

Full RTL file list found under `rtl/`:

```

alu.v
bram_ram.v
bram_rom.v
branch_predictor.v
clint.v
control.v
cpu.v
cpu_core.v
cpu_core_dbg.v
cpu_core_mmu.v
cpu_mc.v
cpu_mc_mmu.v
cpu_pipe.v
cpu_pipe_bp.v
cpu_pipe_trap.v
csr.v
debug_module.v
dmem.v
fpga_top.v
fpga_top_full.v
fpga_top_rtos.v
fpga_top_uart.v
imem.v
immgen.v
mmu.v
regfile.v
soc.v
soc_dbg.v
soc_fpga.v
soc_fpga_mmu.v
soc_mmu.v
soc_pipe.v
soc_rtos.v
soc_rtos_fpga.v
soc_uart_fpga.v
syscon.v
timer.v

```

```
uart.v  
uart_full.v  
uart_hw.v  
uart_rx.v  
uart_tx.v  
uart_tx_cfg.v
```

3 CPU Architecture Overview

3.1 The basic CPU loop

Before looking at individual modules, it helps to understand the life of a typical instruction.

For example:

```
add x3, x1, x2
```

Conceptually, the CPU does this:

1. Use PC to fetch a 32-bit instruction word.
2. Decode that word and discover it is an R-type ADD.
3. Read source registers x1 and x2.
4. Send both values to the ALU.
5. ALU computes $rs1 + rs2$.
6. Write the result into destination register x3.
7. Advance PC to $PC + 4$.

A load instruction adds a memory read. A store adds a memory write. A branch changes the next PC depending on a comparison. A trap redirects the PC to a trap handler.

3.2 Main conceptual block diagram

Listing 1: Mermaid diagram source

```
flowchart LR
    PC[Program Counter] --> IF[Instruction Fetch]
    IF --> DEC[Instruction Decoder]
    DEC --> REG[Register File]
    DEC --> IMM[Immediate Generator]
    REG --> ALU[ALU]
    IMM --> ALU
    ALU --> BR[Branch / Jump Decision]
    ALU --> LSU[Load / Store Address]
    LSU --> MEM[Data Memory / Bus / MMU]
    MEM --> WB[Write Back]
    ALU --> WB
    WB --> REG
    BR --> PC
    DEC --> CTRL[Control Signals]
    CTRL --> ALU
    CTRL --> LSU
    CTRL --> WB
```

3.3 Single-cycle vs multi-cycle vs pipelined

The repository contains all three styles.

3.3.1 Single-cycle

In a single-cycle CPU, one instruction completes in one clock period. The clock period must be long enough for the worst instruction path: instruction memory, decode, register read, ALU, memory, write-back mux, and setup to the register file.

This is easy to understand but inefficient for FPGA block RAM and high clock frequency. In this project, `rtl/cpu.v` is the cleanest self-contained single-cycle CPU, while `rtl/cpu_core.v` exposes the data-memory side as an external bus for SoC use.

3.3.2 Multi-cycle

In a multi-cycle CPU, the same instruction is split across several clock cycles. The CPU uses a small finite-state machine: fetch the instruction, execute it, optionally access memory, then return to fetch. This is slower in instructions per clock but easier to map to registered memories.

`rtl/cpu_mc.v` is this version. The comments describe states such as `FETCH`, `EXEC`, and `MEM`: ALU/branch/store instructions take fewer states than loads, because loads need an extra memory-data cycle.

3.3.3 Pipelined

In a pipelined CPU, multiple instructions are in flight at once. A classic five-stage pipeline uses:

```
IF  -> instruction fetch
ID  -> decode and register read
EX  -> execute / branch decision / address calculation
MEM -> data memory access
WB  -> register write-back
```

`rtl/cpu_pipe.v` implements this style. It improves throughput, but introduces hazards: one instruction may need a value that an older instruction has not written back yet, or the CPU may fetch down the wrong path after a branch.

4 Instruction Encoding and Decode

4.1 Why decode exists

Hardware does not see assembly text like `add x3, x1, x2`. It sees a 32-bit instruction word. Decode is the process of slicing that word into fields and turning those fields into control signals.

RISC-V keeps register fields in fixed positions across most instruction formats:

```
rd      = instr[11:7]
funct3  = instr[14:12]
rs1     = instr[19:15]
rs2     = instr[24:20]
funct7  = instr[31:25]
opcode  = instr[6:0]
```

The opcode identifies the broad instruction class: R-type ALU, I-type ALU, load, store, branch, jump, system, and so on. `funct3` and `funct7` refine that class.

4.2 How decode is implemented here

The main decoder/control unit is `rtl/control.v`. It receives:

```
opcode
funct3
funct7
```

and produces control signals such as:

Control signal	Meaning
<code>reg_write</code>	Write a value to <code>rd</code> at the end of the instruction.
<code>alu_src_a</code>	Select ALU operand A: register value or PC.
<code>alu_src_b</code>	Select ALU operand B: register value or immediate.
<code>mem_read</code>	The instruction reads from data memory.
<code>mem_write</code>	The instruction writes to data memory.
<code>branch</code>	The instruction is a conditional branch.
<code>jump</code>	The instruction is <code>jal</code> or <code>jalr</code> .
<code>jalr</code>	The jump target is computed from <code>rs1 + imm</code> and bit 0 is cleared.
<code>wb_sel</code>	Selects write-back source: ALU, memory, PC+4, or immediate.
<code>imm_type</code>	Selects I/S/B/U/J immediate layout.
<code>alu_op</code>	Selects the ALU operation.

The decoder uses the common hardware style “defaults first, then override”. Unknown instructions default to doing nothing in the simple decoder; trap-capable CPU variants add illegal-instruction detection around this.

4.3 Instruction families

4.3.1 R-type integer operations

R-type instructions use two registers as operands and write one register:

```
add/sub/and/or/xor/sll/srl/sra/slt/sltu  
mul/div/rem variants when RV32M is enabled
```

In `control.v`, opcode 0110011 selects R-type behavior. If `funct7 == 0000001`, the instruction is treated as an RV32M multiply/divide operation. Otherwise, `funct3` and `funct7[5]` select the base integer ALU operation.

4.3.2 I-type ALU operations

I-type ALU instructions use one register and one immediate:

```
addi/slti/sltiu/xori/ori/andi/slli/srli/srai
```

The decoder sets `alu_src_b = 1` so the ALU receives the immediate instead of `rs2`.

4.3.3 Loads and stores

Loads and stores use the ALU as an address adder:

```
effective_address = rs1 + sign_extended_immediate
```

Loads write memory data back to `rd`. Stores write `rs2` into memory and do not write a register.

4.3.4 Branches and jumps

Branches compare two register values and may redirect the PC to `PC + branch_offset`. Jumps always redirect the PC and write `PC + 4` to `rd` as a return address.

4.3.5 System instructions

Trap-capable variants look for opcode 1110011 and distinguish CSR instructions from `ecall`, `ebreak`, and `mret`. This logic appears in CPU wrappers such as `cpu_core.v`, `cpu_mc.v`, `cpu_pipe_trap.v`, and MMU variants, while the CSR state lives in `rtl/csr.v`.

5 Immediate Generator

5.1 Why immediates look strange in RISC-V

Many RISC-V instructions contain a constant value, called an immediate. For example:

```
addi x1, x0, -1
lw   x5, 12(x1)
beq  x1, x2, label
```

A beginner might expect every immediate to occupy one contiguous field in the instruction word. RISC-V does not do that. Instead, it keeps important fields such as `rd`, `rs1`, `rs2`, and the sign bit in stable positions. This simplifies register decode and sign extension, but it means the immediate generator must reassemble scattered bits.

5.2 Immediate formats

`rtl/immgen.v` implements five immediate types:

Type	Used by	Meaning
I	ALU-immediate, loads, <code>jalr</code> , CSR immediate-style decode support	12-bit sign-extended immediate from <code>instr[31:20]</code> .
S	Stores	Immediate split between <code>instr[31:25]</code> and <code>instr[11:7]</code> .
B	Conditional branches	Branch offset, includes an implicit low zero bit.
U	<code>lui</code> , <code>auipc</code>	Upper 20 bits, lower 12 bits zero.
J	<code>jal</code>	Jump offset, includes an implicit low zero bit.

5.3 Why B and J immediates end in zero

Branch and jump targets are instruction addresses. In this design, instructions are 32-bit aligned for the normal RV32I path, so the target is at least even. RISC-V encodes branch and jump offsets with an implicit low zero bit, which gives a larger address range without storing a bit that is normally zero.

For a branch, the reconstructed immediate is:

```
{{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}
```

This looks unusual, but it simply means: collect the scattered branch offset bits, put a zero in bit 0, and sign-extend from the sign bit.

5.4 Implementation role

The immediate generator does not know whether an instruction is valid. It only receives an `imm_type` from the control unit and produces a 32-bit value. The CPU then decides

how to use that value: as an ALU operand, branch offset, jump offset, store offset, or upper immediate.

6 Register File

6.1 What the register file is

The register file is the CPU's small, fast architectural storage. RISC-V RV32 has 32 integer registers:

```
x0, x1, x2, ..., x31
```

Each is 32 bits wide in RV32. Most instructions read one or two registers and write one destination register.

6.2 The special rule for x0

RISC-V register x0 is hardwired to zero. Reading x0 always returns zero. Writing x0 has no effect. This simplifies many instructions. For example:

```
addi x5, x0, 123    # load small constant 123 into x5
add  x6, x7, x0      # copy x7 into x6
```

`rtl/regfile.v` implements this rule by returning zero when a read address is zero and ignoring writes to `rd_addr == 0`.

6.3 Read and write timing

The register file has:

- two combinational read ports: `rs1_data` and `rs2_data` update when addresses change;
- one synchronous write port: writes occur on the rising clock edge.

This is convenient for the single-cycle CPU because operands can be read, executed, and scheduled for write-back within one clock period. In the pipelined CPU, register-file timing interacts with hazards, forwarding, and write-back bypassing.

7 ALU and RV32M Multiply/Divide

7.1 Why the ALU exists

The ALU is the CPU's arithmetic and logic engine. It performs integer operations and address calculations. Even memory instructions use the ALU: a load or store address is computed as `rs1 + immediate`.

`rtl/alu.v` is combinational. It receives two operands, `a` and `b`, plus a 5-bit `alu_op`, and produces `result` plus a `zero` flag.

7.2 Base RV32I ALU operations

The base operations implemented include:

Operation	Meaning	Used by
ADD	$a + b$	<code>add</code> , <code>addi</code> , load/store address, <code>auipc</code>
SUB	$a - b$	<code>sub</code> , branch comparison support
AND	bitwise and	<code>and</code> , <code>andi</code>
OR	bitwise or	<code>or</code> , <code>ori</code>
XOR	bitwise xor	<code>xor</code> , <code>xori</code>
SLL	logical left shift	<code>sll</code> , <code>slli</code>
SRL	logical right shift	<code>srl</code> , <code>srli</code>
SRA	arithmetic right shift	<code>sra</code> , <code>srai</code>
SLT	signed less-than	<code>slt</code> , <code>slti</code>
SLTU	unsigned less-than	<code>sltu</code> , <code>sltiu</code>

For shifts, only `b[4:0]` is used. A 32-bit value only needs shift amounts 0 through 31.

7.3 Signed vs unsigned behavior

The same 32-bit bit pattern can be interpreted as signed or unsigned. For example, `0xFFFF_FFFF` is:

- unsigned: 4,294,967,295
- signed two's-complement: -1

That is why the ALU has both `SLT` and `SLTU`, and both signed and unsigned division/remainder in RV32M. The Verilog uses `$signed(...)` where signed interpretation is required.

7.4 RV32M multiplication

RV32M adds multiply instructions. A 32-bit by 32-bit multiply naturally produces a 64-bit result. RISC-V exposes different parts of that result:

Instruction	Meaning
<code>mul</code>	Low 32 bits of signed $\times$ signed product.
<code>mulh</code>	High 32 bits of signed $\times$ signed product.
<code>mulhsu</code>	High 32 bits of signed $\times$ unsigned product.
<code>mulhu</code>	High 32 bits of unsigned $\times$ unsigned product.

`rtl/alu.v` creates 64-bit signed and unsigned products (`p_ss`, `p_su`, `p_uu`) and selects either the low or high half.

7.5 RV32M division and remainder

Division has architectural corner cases. The implementation handles the important RV32M-defined cases:

Case	<code>div</code> result	<code>rem</code> result
Divide by zero	<code>0xFFFF_FFFF</code>	dividend <code>a</code>
Signed overflow: <code>INT_MIN / -1</code>	<code>0x8000_0000</code>	0

Unsigned division by zero also returns all ones for quotient and the dividend for remainder.

7.6 Hardware cost

The current ALU uses Verilog `*`, `/`, and `%` for clarity. Multiplication usually maps well to FPGA DSP resources. Division, however, can be large if implemented as a purely combinational block. A production CPU commonly makes division multi-cycle, using an iterative divider and stall/ready logic.

So the design is excellent for learning and simulation, but the divider is an obvious target for refinement in a frequency/area-optimized FPGA version.

8 Program Counter, Branches, and Jumps

8.1 What the PC does

The program counter is the address of the current instruction. For ordinary instructions, the next PC is:

```
next_pc = pc + 4
```

because base RV32 instructions are 4 bytes long.

8.2 Conditional branches

A branch instruction compares two registers. If the condition is true, the CPU jumps to $PC + \text{immediate}$. Otherwise, it falls through to $PC + 4$.

The branch conditions implemented in the CPU variants include:

funct3	Branch	Meaning
000	beq	branch if equal
001	bne	branch if not equal
100	blt	signed less-than
101	bge	signed greater/equal
110	bltu	unsigned less-than
111	bgeu	unsigned greater/equal

The branch comparison is implemented in the CPU wrapper logic, not as a separate source file. In the pipelined CPU, branch comparison uses forwarded operands so a branch can depend on recently computed values.

8.3 8.3 jal

jal is an unconditional PC-relative jump. It also writes a return address:

```
rd = PC + 4  
PC = PC + J-immediate
```

This supports function calls. Typically `rd` is `x1 (ra)`.

8.4 8.4 jalr

jalr jumps to an address computed from a register plus an immediate:

```
target = rs1 + imm  
PC = target with bit 0 cleared  
rd = PC + 4
```

The design computes the target through the ALU and clears bit 0 with:

```
{alu_result[31:1], 1'b0}
```

Clearing bit 0 is required by RISC-V and helps support pointer tagging and alignment behavior.

8.5 Control hazard in a pipeline

In a single-cycle CPU, the branch decision is known before the clock edge updates the PC. In a pipeline, the fetch stage may already have fetched one or two younger instructions before the branch reaches execute. If the branch is taken, those younger instructions are wrong-path instructions and must be flushed.

This is why `cpu_pipe.v` flushes behind taken branches and jumps, and why `cpu_pipe_bp.v` later adds prediction to avoid paying that cost every time.

9 Load/Store Unit and Memory Access

9.1 Why RISC-V is load/store

RISC-V is a load/store architecture. Arithmetic instructions operate on registers, not directly on memory. To use a memory value, software must first load it into a register. To update memory, software must store a register value.

```
lw    x5, 0(x10)    # x5 = memory[x10 + 0]
add   x6, x5, x7     # arithmetic uses registers
sw    x6, 4(x10)     # memory[x10 + 4] = x6
```

9.2 Effective address calculation

Loads and stores compute an effective address:

```
effective_address = rs1 + sign_extended_immediate
```

The ALU performs this addition. In simple cores, this address goes directly to `dmem.v`. In SoC-oriented cores, it goes to an external data bus. In MMU variants, it may first be translated from virtual to physical.

9.3 Load sizes and sign extension

RISC-V loads can read different sizes:

Instruction	Size	Extension
lb	8-bit byte	sign-extend
lbu	8-bit byte	zero-extend
lh	16-bit halfword	sign-extend
lhu	16-bit halfword	zero-extend
lw	32-bit word	no extension needed

The memory block or CPU memory stage must select the correct byte/halfword lane and extend it to 32 bits.

9.4 Store sizes and byte masks

Stores can write bytes, halfwords, or words:

Instruction	Size
sb	8-bit byte
sh	16-bit halfword
sw	32-bit word

The multi-cycle memory design uses byte write masks such as `4'b0001 << byte_offset` for byte stores and similar masks for halfword/word stores. This is important for block RAM and bus integration.

9.5 Memory-mapped I/O

In `cpu_core.v`, the CPU does not own data memory directly. Instead it exposes:

```
dmem_addr  
dmem_wdata  
dmem_we  
dmem_funct3  
dmem_rdata
```

A SoC wrapper decodes the address and decides whether the access targets RAM, UART, timer, syscon, or another peripheral. This is memory-mapped I/O: software uses normal load/store instructions, but some addresses control hardware devices instead of RAM.

10 CSR, Privilege, Exceptions, and Traps

10.1 Why CSRs exist

General-purpose registers hold normal program data. A CPU also needs control/status registers for privileged state: interrupt enables, trap handler address, trap cause, saved exception PC, timers, and address translation control.

RISC-V calls these **CSRs**: Control and Status Registers.

10.2 What a trap is

A trap is a controlled redirection of execution caused by an exception or interrupt.

- An **exception** is caused by the current instruction, such as illegal instruction or `ecall`.
- An **interrupt** is caused by an external/asynchronous event, such as a timer.

On a trap, the CPU must save enough information so software can handle the event and later return.

Conceptually:

```
mepc    = PC of interrupted/faulting instruction
mcause  = reason for trap
PC      = mtvec
mode    = machine mode
```

10.3 Privilege model in this design

`rtl/csr.v` implements a minimal privilege model:

Mode	Encoding	Meaning
U-mode	2'b00	User mode, restricted.
M-mode	2'b11	Machine mode, privileged.

The design resets into machine mode. Traps enter machine mode. `mret` restores the previous privilege mode using bits saved in `mstatus`.

10.4 Supported CSRs

`rtl/csr.v` supports:

CSR	Address	Role
<code>mstatus</code>	0x300	Global interrupt enable and previous interrupt enable bits.
<code>mie</code>	0x304	Machine interrupt enable bits, including timer/external.

CSR	Address	Role
mtvec	0x305	Trap handler base address.
mscratch	0x340	Scratch register for trap handlers.
mepc	0x341	Saved PC for trap return.
mcause	0x342	Trap cause.
mip	0x344	Pending interrupt bits, read-only style here.
satp	0x180	Address translation control for MMU variants.

10.5 CSR instructions

The CSR unit supports the CSR read/write family through the `csr_next` function:

Operation	Meaning
csrrw / csrrwi	Write source value into CSR.
csrrs / csrrsi	Set CSR bits selected by source.
csrrc / csrrci	Clear CSR bits selected by source.

The CPU writes the old CSR value to `rd` and updates the CSR according to the instruction.

10.6 10.6 Trap entry and mret

`csr.v` prioritizes updates as:

```
reset > trap entry > mret > CSR write
```

On trap entry:

- `mepc` receives the trap PC;
- `mcause` receives the cause;
- `mstatus.MPIE` receives old MIE;
- `mstatus.MIE` is cleared to disable nested interrupts by default;
- current privilege is saved into `mstatus.MPP`;
- current privilege becomes machine mode.

On `mret`:

- MIE is restored from `MPIE`;
- privilege is restored from `MPP`;
- `MPP` is cleared down toward user mode;
- PC redirects to `mepc`.

10.7 Illegal instruction detection

Trap-capable CPU wrappers compute whether the opcode/instruction is known. If not, they raise illegal instruction cause 2. They also treat user-mode attempts to execute privileged instructions such as CSR access or `mret` as privilege violations.

11 Timer and CLINT

11.1 Why a timer matters

A timer interrupt lets software regain control periodically. This is essential for:

- preemptive multitasking;
- RTOS tick interrupts;
- timeouts;
- scheduling;
- basic OS services.

Without a timer, one program can run forever unless it voluntarily yields.

11.2 Machine timer interrupt flow

The timer peripheral asserts a timer interrupt level. The CSR unit checks:

```
mstatus.MIE && mie.MTIE && timer_irq
```

If true, `irq_pending` becomes active and the CPU takes a machine timer interrupt. The CSR unit reports cause `0x8000_0007` for timer interrupt. External interrupts, such as UART-style interrupts in later variants, use cause `0x8000_000B` and have priority in `csr.v`.

11.3 CLINT-style block

`rtl/clint.v` is the CLINT-style timer block used by RTOS-oriented SoC variants. In RISC-V systems, CLINT normally contains `mtime` and `mtimecmp`. When `mtime >= mtimecmp`, a timer interrupt becomes pending.

The important concept is that software programs a compare value in memory-mapped registers, then the hardware raises an interrupt when time reaches that value.

12 Atomic Operations

12.1 Why atomics exist

Consider this code running with interrupts or multiple execution contexts:

```
counter = counter + 1;
```

At machine level, that is usually:

```
lw    t0, 0(a0)
addi  t0, t0, 1
sw    t0, 0(a0)
```

If an interrupt or another hart modifies `counter` between the load and store, an update can be lost. Atomic instructions solve this by making read-modify-write sequences indivisible at the architectural level.

12.2 RISC-V atomic concepts

RISC-V atomics include two styles:

1. **AMO instructions:** read old memory value, compute a new value, write it back, and return the old value.
2. **LR/SC instructions:** load-reserved and store-conditional. `lr.w` creates a reservation. `sc.w` stores only if the reservation is still valid.

12.3 Implementation in this repository

`cpu_core.v` contains atomic datapath logic for opcode 0101111:

- `funct7[6:2]` is used as the AMO function field;
- `lr.w` records a reservation address;
- `sc.w` succeeds only if the reservation is valid and matches the address;
- AMO read-modify-write instructions compute `amo_alu` from memory data and `rs2`;
- atomics are word-sized in this design (`dmem_funct3 = 3'b010`).

The reservation state is held in:

```
resv_valid
resv_addr
```

The design clears the reservation after store-conditional and AMO behavior, and also clears it on traps in the relevant logic. That matters because a task switch between `lr.w` and `sc.w` should normally cause the later `sc.w` to fail.

12.4 Important limitation

The atomic path is present in the SoC-style core logic, not uniformly integrated into every pipeline and MMU variant. A pipelined implementation would need to ensure the read-modify-write sequence is not split incorrectly across stages or interrupted by hazards.

13 MMU and Virtual Memory

13.1 The problem: software should not always use physical addresses

Without an MMU, every address produced by software is a physical address. If code executes:

```
sw x5, 0(x10)
```

and `x10` contains `0x10000000`, then the CPU writes physical address `0x10000000` directly. That address might be RAM, a UART register, a timer register, another program's memory, or an invalid area.

This is simple, but it has problems:

- programs can corrupt each other;
- user software can access device registers directly;
- each program must know where it is physically loaded;
- operating systems cannot easily isolate processes;
- memory can be fragmented and still needs to appear contiguous to software.

13.2 Virtual address vs physical address

Virtual memory introduces a translation step:

software address	->	MMU translation	->	physical memory/bus address
virtual address		page table walk		physical address

The program produces a **virtual address**. The memory system needs a **physical address**. The MMU translates between them.

This lets the operating system give each program its own address space. Two programs can both think they use address `0x00001000`, but the MMU can translate those virtual addresses to different physical pages.

13.3 What a page is

A page is a fixed-size block of virtual memory. In RISC-V Sv32, the normal page size is 4 KiB. This means the lower 12 bits of an address are the offset within the page:

```
4 KiB = 4096 bytes = 2^12 bytes
```

So address bits [11:0] are not translated. They are copied into the physical address. The upper bits select which page is being accessed.

13.4 What a page table is

A page table is a data structure in memory that tells the MMU how to translate virtual pages into physical pages. Each entry is called a PTE: Page Table Entry.

A PTE contains:

- a physical page number;

- valid/read/write/execute/user permission bits;
- other status bits.

The MMU reads PTEs from memory during a page-table walk.

13.5 Why multi-level page tables exist

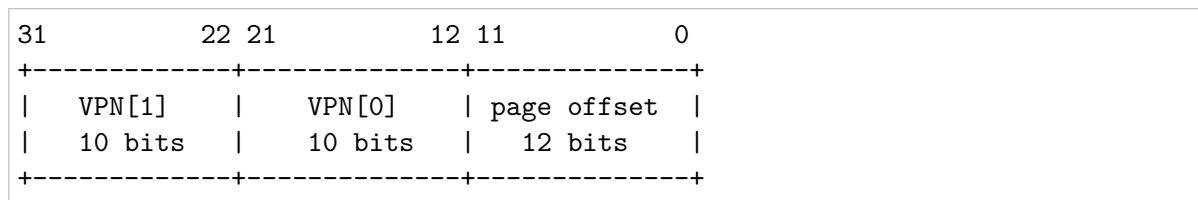
A flat page table for a 32-bit address space could be large. Multi-level page tables solve this by splitting the virtual page number into pieces. The first piece indexes a root table. That entry either points to a second-level table or directly maps a large page. The second piece indexes the second-level table.

This saves memory because page tables only need to exist for regions that are actually mapped.

13.6 Now Sv32: the RISC-V 32-bit virtual memory scheme

Only after understanding pages and page tables does Sv32 make sense.

Sv32 is the RISC-V virtual memory scheme for 32-bit virtual addresses. It divides the address like this:

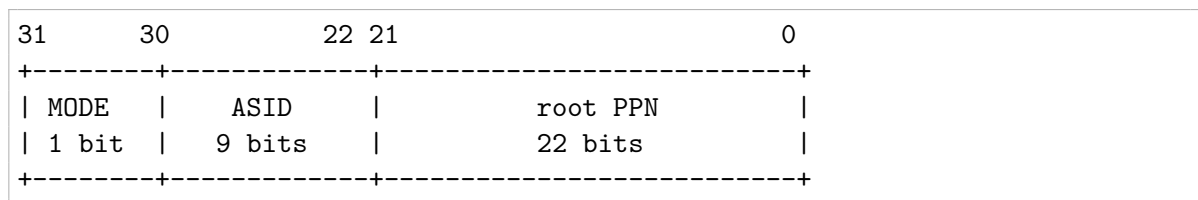


- VPN[1] indexes the level-1/root page table.
- VPN[0] indexes the level-0 page table.
- page offset is copied to the physical address.

13.7 13.7 satp: where translation begins

The CPU needs to know where the root page table is. RISC-V stores that information in the `satp` CSR.

For RV32 Sv32, the useful layout is:



In this implementation:

- `satp[31] = 1` enables Sv32 translation;
- `satp[21:0]` is used as the root page table physical page number;
- ASID is structurally present in the format but not used for a TLB because this educational MMU has no TLB.

13.8 PTE format and permissions

`rtl/mmu.v` uses these PTE bits:

Bit	Name	Meaning
0	V	Valid
1	R	Readable
2	W	Writable
3	X	Executable
4	U	User-accessible
5	G	Global, noted but not central here
6	A	Accessed, noted but not updated here
7	D	Dirty, noted but not updated here
31:10	PPN	Physical page number

The implementation treats a PTE as invalid if V=0 or if R=0 and W=1, which is an invalid combination.

13.9 Page-table walk

For a data access in user mode when Sv32 is enabled, the MMU performs:

```
pte1_addr = satp.root_ppn * 4096 + VPN[1] * 4
pte1 = memory[pte1_addr]

if pte1 is a leaf:
    map as a 4 MiB superpage
else:
    pte0_addr = pte1.ppn * 4096 + VPN[0] * 4
    pte0 = memory[pte0_addr]
    pte0 must be a leaf mapping a 4 KiB page
```

A leaf is detected when R or X is set. A level-1 leaf maps a superpage. A level-0 leaf maps a normal 4 KiB page.

13.10 How physical addresses are formed

For a 4 KiB page:

```
physical_address = pte0.PPN || virtual_address[11:0]
```

For a 4 MiB superpage:

```
physical_address = pte1.PPN[1] || virtual_address[21:0]
```

`rtl/mmu.v` implements these as `pa_4k` and `pa_super`.

13.11 When translation is active

In `rtl/mmu.v`, translation is active only when:

```
priv == user mode AND satp[31] == 1
```

Machine mode bypasses translation and uses addresses directly. This keeps machine-mode trap handlers and boot code simple.

13.12 Page faults

The MMU reports a page fault when:

- a PTE is invalid;
- a required second-level leaf is missing;
- user mode accesses a page without `U=1`;
- a load is attempted without read permission;
- a store is attempted without write permission.

The CPU converts this into RISC-V trap causes:

Cause	Meaning
13	Load page fault
15	Store/AMO page fault

13.13 Two MMU implementations in this project

There are two MMU styles:

13.13.1 `rtl/mmu.v` — combinational educational MMU

This module has dedicated page-table read ports:

```
walk_addr1 / walk_data1  
walk_addr2 / walk_data2
```

That lets the page walk complete combinational in one cycle. This is easy to understand and useful for simulation, but unrealistic for a single-port block RAM system.

13.13.2 `rtl/cpu_mc_mmu.v` — synthesizable multi-cycle walker

This version sequences the walk through states such as:

```
PTW1  -> drive level-1 PTE address  
PTW1D -> receive/check level-1 PTE  
PTW0D -> receive/check level-0 PTE  
ACC   -> perform translated memory access  
ACCD  -> receive load data  
PF    -> raise page-fault trap
```

This is closer to real hardware because the page-table walker shares the memory/data port and waits for registered memory data.

13.14 What is simplified

This MMU is educational. It does not implement a TLB, does not update Accessed/Dirty bits, and focuses on data-side translation. A production MMU would add instruction-side translation, TLBs, permission details such as SUM/MXR, page-fault priority details, and more complete privilege behavior.

14 Pipeline, Hazards, Forwarding, and Stalls

14.1 Why pipelining exists

A single-cycle CPU waits for every part of an instruction to finish before starting the next instruction. Pipelining improves throughput by overlapping stages from different instructions.

An ideal five-stage pipeline completes one instruction per cycle after filling:

```
Cycle 1: I1 IF
Cycle 2: I1 ID   I2 IF
Cycle 3: I1 EX   I2 ID   I3 IF
Cycle 4: I1 MEM  I2 EX   I3 ID   I4 IF
Cycle 5: I1 WB   I2 MEM  I3 EX   I4 ID   I5 IF
```

14.2 Pipeline registers

Between stages, the CPU stores values in pipeline registers. `cpu_pipe.v` has registers with names like:

```
IF/ID: ifid_pc, ifid_instr
ID/EX: idex_pc, idex_imm, idex_rs1d, idex_rs2d, control bits
EX/MEM: exmem_addr, exmem_store, exmem_result, control bits
MEM/WB: memwb_data, memwb_rd, memwb_reg_write
```

These registers preserve each instruction's state as it moves through the pipeline.

14.3 Data hazards

A data hazard occurs when a younger instruction needs a result from an older instruction that has not yet reached write-back.

Example:

```
add x5, x1, x2
sub x6, x5, x3
```

The `sub` needs `x5` before the `add` writes it back.

14.4 Forwarding

Forwarding solves many hazards by taking a value from a later pipeline stage and feeding it directly into EX, without waiting for the register file.

`cpu_pipe.v` forwards from:

- EX/MEM to EX;
- MEM/WB to EX.

The forwarding checks compare destination registers against current source registers and ignore register zero.

14.5 Load-use stall

A load result is not available until the memory stage. If the very next instruction needs that loaded value, forwarding cannot provide it soon enough.

Example:

```
lw  x5, 0(x1)
add x6, x5, x2
```

The pipeline inserts a one-cycle stall. In `cpu_pipe.v`, this is detected by checking whether the instruction in EX is a load and its destination matches `rs1` or `rs2` of the instruction in ID.

14.6 Control hazards and flushing

Branches and jumps decide the next PC. In `cpu_pipe.v`, they resolve in EX. If a branch or jump is taken, instructions currently in IF and ID are on the wrong path. The CPU flushes them and redirects the PC to the correct target.

This is called a predict-not-taken pipeline because the fetch unit keeps fetching `PC+4` until EX proves otherwise.

15 Precise Traps in the Pipeline

15.1 Why traps are harder in a pipeline

In a non-pipelined CPU, only one instruction is active, so taking a trap is straightforward. In a pipeline, several instructions are active at once. If an instruction traps, older instructions should already be committed, but younger instructions must not commit.

This property is called a **precise trap**.

15.2 What precise means

A precise trap means the machine state looks as if execution stopped exactly at one instruction boundary:

```
all older instructions completed
trapping instruction did not incorrectly commit partial side effects
younger instructions did not commit
mepc identifies the faulting/interrupted instruction
```

15.3 Implementation in `cpu_pipe_trap.v`

`cpu_pipe_trap.v` commits traps in the EX stage. When the instruction in EX traps:

- the trapping instruction is squashed so it does not write registers or memory;
- younger instructions in IF and ID are flushed;
- `mepc` receives the EX-stage instruction PC;
- `mcause` receives the trap cause;
- PC redirects to `mtvec`.

`mret` redirects back to `mepc`. CSR instructions commit only if the instruction is valid and not trapping.

15.4 Why EX-stage trap commit is a reasonable teaching choice

EX is where branch decisions, ALU exceptions/illegal decisions, CSR operations, and many control decisions are already centralized. Committing traps there keeps the design easier to reason about. More advanced CPUs may detect exceptions in multiple stages and carry exception metadata until commit/retirement.

16 Branch Prediction

16.1 Why prediction exists

In the basic pipeline, every taken branch costs a flush because the CPU guessed the next PC would be $PC+4$. Loops often branch backward many times, so “predict not taken” performs badly on common code.

Branch prediction tries to guess the next PC early, in the fetch stage, before decode and execute know the truth.

16.2 What must be predicted

A predictor needs two answers:

1. **Direction:** will the branch/jump be taken?
2. **Target:** if taken, where should fetch go next?

The CPU only has the current PC during fetch, so the predictor stores past behavior indexed by PC.

16.3 BHT: branch history table

`rtl/branch_predictor.v` contains a BHT with 2-bit saturating counters:

```
00 strong not-taken
01 weak not-taken
10 weak taken
11 strong taken
```

The top bit is the prediction. A two-bit counter avoids changing opinion after one unusual branch outcome. This is especially useful for loops: a loop branch may be taken many times and not taken once when the loop exits.

16.4 BTB: branch target buffer

The BHT predicts direction, but the CPU also needs a target address. The BTB stores:

```
valid bit
tag
target address
```

The predictor only predicts taken when the BTB hits and the BHT counter says taken. If there is no BTB target, fetch cannot safely redirect.

16.5 Training the predictor

Prediction happens combinationally in IF using `pc_if`. Update happens synchronously when a control instruction resolves in EX:

- taken branches increment the counter toward strongly taken;
- not-taken branches decrement toward strongly not-taken;
- jumps are forced to strongly taken;
- taken control transfers allocate/update the BTB target.

16.6 16.6 Integration in `cpu_pipe_bp.v`

`cpu_pipe_bp.v` carries prediction metadata through the pipeline:

```
ifid_pred_taken / ifid_pred_target  
idex_pred_taken / idex_pred_target
```

When the instruction reaches EX, the CPU compares predicted behavior against actual behavior:

```
direction wrong? actual_taken != predicted_taken  
target wrong? actual_taken && predicted_taken && actual_target !=  
    predicted_target
```

If either is wrong, the pipeline flushes younger instructions and redirects to the correct PC.

16.7 Performance counters

The branch-predicting core exposes counters such as:

- total cycles;
- number of control transfers;
- number of taken control transfers;
- number of mispredictions.

These make it possible to compare a predict-not-taken pipeline against the BTB/BHT predictor on the same program.

17 SoC Integration and Memory Map Concept

17.1 Why a CPU core needs a SoC wrapper

A CPU alone only executes instructions. A usable system needs memory and peripherals. The SoC wrapper connects the CPU to:

- instruction memory;
- data RAM;
- UART;
- timer/CLINT;
- syscon/control registers;
- debug blocks;
- FPGA top-level pins.

17.2 Memory-mapped peripherals

The CPU uses normal load/store instructions. The SoC address decoder decides what hardware block responds.

CPU address -> address decoder -> RAM or UART or timer or syscon

This means software can write a UART transmit register with a normal store instruction.

17.3 Important wrappers

File	Role
rtl/soc.v	Basic simulation SoC around <code>cpu_core.v</code> .
rtl/soc_pipe.v	SoC wrapper for pipelined CPU experiments.
rtl/soc_mmu.v	SoC wrapper for MMU experiments.
rtl/soc_fpga.v, rtl/soc_uart_fpga.v, rtl/soc_fpga_mmu.v	FPGA-oriented system wrappers.
rtl/soc_rtos.v, rtl/soc_rtos_fpga.v	RTOS-oriented systems with timer/UART integration.
rtl/fpga_top*.v	Board-level top modules.

18 UART, Debug, and System Control

18.1 Why UART is useful

A UART gives the CPU a simple way to print text, receive commands, and show program progress without a full display or operating system. Bare-metal programs often use UART as their first debug interface.

The repository contains several UART-related modules:

```
uart.v
uart_hw.v
uart_full.v
uart_rx.v
uart_tx.v
uart_tx_cfg.v
```

These are used by FPGA/SoC wrappers to provide serial I/O.

18.2 Debug support

`debug_module.v`, `cpu_core_dbg.v`, and `soc_dbg.v` are debug-oriented blocks/wrappers. The idea of a debug module is to let an external controller observe or influence CPU execution, for example halting, inspecting state, or stepping through code.

This appears experimental/educational rather than a full RISC-V Debug Specification implementation.

18.3 System control

`syscon.v` provides system-level control. In some SoC variants, a halt signal can freeze the CPU. This is useful in simulations, tests, or simple host-controlled systems.

19 FPGA and Synthesis Notes

19.1 Why simulation code and FPGA code differ

A simulation memory can return data combinatorially. FPGA block RAM usually returns data one clock after the address is registered. That difference strongly affects CPU architecture.

This is why the repository has both single-cycle and multi-cycle versions. The single-cycle core is easy to teach. The multi-cycle core is more realistic for FPGA memories.

19.2 FPGA top modules

The FPGA top-level files connect the SoC to board pins and clock/reset resources:

```
fpga_top.v  
fpga_top_full.v  
fpga_top_rtos.v  
fpga_top_uart.v
```

The constraints directory contains Zynq-related `.xdc` files.

19.3 Practical synthesis concerns

The biggest synthesis concerns are:

- combinational divider cost in `alu.v`;
- combinational MMU walk in `mmu.v` if mapped directly;
- memory timing assumptions in single-cycle designs;
- timing closure in pipelined variants with forwarding and branch logic;
- UART clock divisor correctness for the actual FPGA clock.

The multi-cycle MMU and multi-cycle CPU variants are better starting points for FPGA-friendly memory behavior.

20 Software and Verification

20.1 Why testbenches matter

A CPU can appear to run simple code while still having subtle bugs in branch offsets, sign extension, load alignment, forwarding, or trap priority. Testbenches catch these by running known programs and checking architectural results.

The repository includes step-by-step docs and testbenches that validate individual blocks and integrated cores.

20.2 What to test per block

Block	Important tests
ALU	Signed/unsigned comparisons, shifts, multiply high variants, division edge cases.
Immediate generator	Negative immediates, B/J offsets, U immediates.
Register file	x0 hardwired zero, simultaneous reads, write enable.
Control/decode	Every opcode family, RV32M funct7 detection, load/store/branch controls.
Branch logic	Signed and unsigned branch conditions, negative branch offsets.
Load/store	Byte/halfword/word, sign/zero extension, unaligned behavior if defined.
CSR/trap	ecall , ebreak , illegal instruction, timer interrupt, mret .
Pipeline	Forwarding, load-use stall, branch flush, write-back hazards.
Branch predictor	Correctness with mispredictions, performance counters, BTB aliasing behavior.
MMU	Valid translations, invalid PTEs, permission faults, superpages, user/M-mode bypass.
Atomics	AMO old/new values, LR/SC success/failure, reservation clearing.

20.3 Software demos

The `sw/` directory contains generated hex files and software experiments. A typical flow is:

```
compile/link bare-metal C or assembly
convert ELF/program to hex
load hex into instruction memory
run simulation or FPGA system
```

observe memory/UART/testbench outputs

This makes the CPU verifiable at the architectural level, not only as isolated HDL blocks.

21 How to Read the Codebase

A good reading order is:

1. `rtl/alu.v` — understand basic operations.
2. `rtl/regfile.v` — understand architectural registers.
3. `rtl/immgen.v` — understand immediate reconstruction.
4. `rtl/control.v` — understand instruction classes and control signals.
5. `rtl/cpu.v` — see the complete single-cycle datapath.
6. `rtl/cpu_core.v` and `rtl/soc.v` — see the CPU become part of a system.
7. `rtl/csr.v` — understand traps, interrupts, and privilege.
8. `rtl/cpu_mc.v` — understand state-machine sequencing and BRAM-friendly timing.
9. `rtl/cpu_pipe.v` — understand pipeline registers, forwarding, stalls, and flushes.
10. `rtl/cpu_pipe_trap.v` — understand precise traps in a pipeline.
11. `rtl/branch_predictor.v` and `rtl/cpu_pipe_bp.v` — understand speculation.
12. `rtl/mmu.v`, `rtl/cpu_core_mmu.v`, and `rtl/cpu_mc_mmu.v` — understand virtual memory.
13. `rtl/soc_*` and `rtl/fpga_top*` — understand integration and deployment.

22 Design Philosophy and Limitations

22.1 Design philosophy

The design favors clarity and incremental learning. Each advanced feature is introduced in a separate variant rather than hidden inside one large, configurable CPU.

This is a good approach for documentation because it lets readers compare:

- single-cycle vs multi-cycle timing;
- non-pipelined vs pipelined hazards;
- predict-not-taken vs dynamic branch prediction;
- physical addressing vs MMU translation;
- simple trap handling vs precise pipelined traps.

22.2 Known simplifications

Important simplifications include:

- combinational divider in `alu.v`;
- educational combinational MMU in `mmu.v`;
- no TLB;
- limited privilege model;
- incomplete production-grade CSR set;
- atomics not uniformly integrated into every variant;
- instruction-side MMU behavior is not as complete as a production Linux-capable core;
- branch predictor is small and direct-mapped by PC index;
- no cache hierarchy.

These are reasonable for a from-scratch learning CPU. They also clearly identify future improvement points.

23 Suggested Future Improvements

Possible next engineering steps:

1. Replace combinational division with a multi-cycle divider.
2. Add a formal instruction decoder table and explicit illegal-instruction trap for all variants.
3. Add instruction-side MMU translation.
4. Add a small TLB for MMU variants.
5. Add A/D bit management for page tables.
6. Integrate atomics into pipelined and MMU variants.
7. Add cache or simple instruction/data buffers.
8. Add a standard bus protocol such as Wishbone, AXI-Lite, or AHB-Lite.
9. Expand CSR/privilege support toward a more complete privileged architecture subset.
10. Add formal verification for ALU, register file, decoder, and pipeline hazards.
11. Add a machine-readable ISA support matrix per CPU variant.
12. Improve debug support toward standard RISC-V debug behavior.

24 Summary

This repository implements a compact but broad RISC-V CPU learning platform. The core starts from the essential RV32 datapath — PC, instruction fetch, decode, immediate generation, register file, ALU, memory access, and write-back — and then adds increasingly realistic CPU features:

- RV32M multiply/divide;
- CSR and trap handling;
- timer interrupts;
- atomics;
- multi-cycle BRAM-friendly execution;
- five-stage pipelining;
- forwarding/stall/flush logic;
- precise pipelined traps;
- branch prediction;
- Sv32-style data MMU translation;
- SoC/FPGA/RTOS integration.

The most important thing to understand is that each module exists to solve a specific architectural problem. `alu.v` computes values. `control.v` turns instruction bits into datapath decisions. `immgen.v` reconstructs constants. `regfile.v` holds architectural registers. `csr.v` gives software controlled privileged state. `mmu.v` translates virtual addresses. `branch_predictor.v` guesses future control flow. The CPU variants show how these same ideas change as the timing model evolves from single-cycle to multi-cycle to pipelined execution.